

TEMA 6 — GENERICIDAD

1) Por qué aparece la genericidad

En programación es muy común querer reutilizar una estructura o algoritmo (lista, pila, par, función de selección, etc.) con tipos distintos sin reescribirlo cada vez (una lista de enteros, otra de strings, otra de puntos...).

Cuando no se usan genéricos, se recurre a “tipos universales”:

En C: void para guardar direcciones de cualquier cosa.

*En Java: Object para guardar cualquier objeto.

Esto da flexibilidad, pero trae problemas importantes: el compilador ya no puede asegurar que lo que sacas es del tipo correcto.

2) Simular “cualquier tipo” con void* (C) o Object (Java)

a) C con void*

Una estructura basada en void* puede almacenar direcciones a datos de tipos diferentes.

El coste es que:

Al recuperar, hay que convertir (cast) al tipo real.

El programador debe “recordar” qué tipo metió.

Si haces un cast incorrecto, el resultado puede ser comportamiento indefinido (errores difíciles).

b) Java con Object

Un array o colección de Object puede guardar cualquier objeto.

El coste es que:

Al recuperar, necesitas casting ((String), (Integer), etc.).

Si el cast no coincide, falla en ejecución con ClassCastException.

Además, puedes mezclar tipos sin querer y el compilador no te protege.

Idea clave: `void*/Object` no es genericidad real, es una simulación que pierde seguridad.

3) Qué es programación genérica (generic programming)

La programación genérica consiste en escribir código parametrizado por tipos, es decir:

Defino una clase o método “en general”

y cuando lo uso, especifico el tipo concreto.

Ventajas centrales:

Reutilización: una sola implementación para muchos tipos.

Chequeo de tipos en compilación: el compilador detecta errores antes de ejecutar.

No necesitas casting al recuperar valores.

Código más expresivo: se ve qué tipo contiene una estructura.

4) Parámetros de tipo (type parameters)

Un parámetro de tipo es un “tipo-variable” como `T`, `U`, `R`, etc.

Ejemplo conceptual:

`Caja<T>` contiene un valor `T`

El compilador garantiza que solo meto/saco `T`.

5) Java Generics vs C++ Templates (cómo difieren “por dentro”)

Ambos permiten colecciones tipadas (lista de `String` con seguridad).

Pero internamente son distintos:

a) C++ (templates): instanciación

El compilador genera una versión concreta del código por cada tipo usado.

Si usas `vector<int>` y `vector<string>`, se generan dos implementaciones.

Mantiene el tipo concreto con precisión (en compilación y ejecución).

Puede producir más código y más coste de compilación.

b) Java (generics): type erasure

En Java, el compilador comprueba tipos, pero luego borra la información genérica.

En ejecución, el tipo genérico concreto no “existe” tal cual.

Normalmente se reemplaza por `Object` o por el límite superior si lo hay.

Consecuencia: ciertas cosas no se pueden saber en runtime sobre `T`.

6) Genéricos en métodos (métodos genéricos)

Un método puede declarar su propio parámetro de tipo, aunque la clase no sea genérica.

Ejemplo conceptual: “elige uno de dos valores del mismo tipo”

Con `Object`: devuelve `Object` → obliga a casting y permite mezclar tipos.

Con `<T>`: devuelve `T` → sin casting y obliga a que ambos argumentos sean del mismo tipo.

Idea clave: el método genérico refuerza dos cosas a la vez:

El retorno es del tipo correcto sin cast.

Los parámetros quedan vinculados a un mismo tipo `T`.

7) Restricciones en parámetros de tipo (bounded generics)

A veces un genérico debe cumplir condiciones.

Ejemplo: “quiero números para poder tratarlos como números”.

Se hace con un límite:

T extends Number

Eso significa:

T debe ser Number o subtipo (Integer, Double, etc.)

Puedes usar operaciones comunes (como convertir a double con doubleValue()).

Comparación típica de dos enfoques

- Usar directamente Number en atributos:
- Usar Punto<T extends Number>:

Con type erasure, si el límite es Number, en ejecución el genérico se trata como Number.

8) Evitar instanceof y downcasting con genéricos

Si una interfaz o clase se diseña con generics, puedes expresar “compatibilidad” en tipos:

Por ejemplo, que un punto 2D solo calcule distancia con otro punto 2D

y un punto 3D solo con otro 3D.

Así el compilador impide mezclas incompatibles sin instanceof ni casting.

Idea clave: expresar restricciones con tipos evita verificaciones manuales.

9) Varianza: arrays vs genéricos

Aunque String sea subtipo de Object, eso no implica lo mismo para contenedores.

Arrays (covariantes)

En Java, String[] sí puede tratarse como Object[].

Pero tiene un riesgo: puedes intentar meter un Integer en un array que realmente es String[] y fallará en ejecución (error de almacenamiento).

Genéricos (invariantes)

List<String> no es subtipo de List<Object>.

Esto se hace para que el compilador evite errores que, si se permitieran, romperían la seguridad de tipos.

Definiciones:

Covariante: si $A \leq B$ entonces $\text{Gen}\langle A \rangle \leq \text{Gen}\langle B \rangle$.

Contravariante: relación “al revés” respecto al parámetro.

Invariante: no hay relación de subtipo entre $\text{Gen}\langle A \rangle$ y $\text{Gen}\langle B \rangle$ aunque $A \leq B$.

10) Wildcards y PECS

Los wildcards permiten flexibilidad controlada en genéricos.

extends T

La colección contiene “algún subtipo de T”.

Útil cuando vas a leer (produce valores).

No es apropiado para añadir elementos porque no sabes cuál es el subtipo exacto.

super T

La colección contiene “algún supertipo de T”.

Útil cuando vas a escribir/añadir valores de tipo T.

Al leer, normalmente solo garantizas Object.

Regla PECS:

Producer → extends

Consumer → super